Chapter 2. Plan to Build Less

> There’s always more to build than you have people, time, or money for. Always.

Speaking in absolutes like “always” and “never” always gets me in trouble. But with the statement above, I honestly can’t recall a situation where it wasn’t true although I have no sciencey data to back this up. No one has ever once come to me saying, “We were asked to add this new feature, and happily we had lots more time than we needed.”

But one of the coolest things about using a story map is that it gives you and other collaborators a space to think through alternatives and to find a way to get a great outcome in the time that you have.

Grab a cup of coffee and settle into your chair. It’s time for a story.

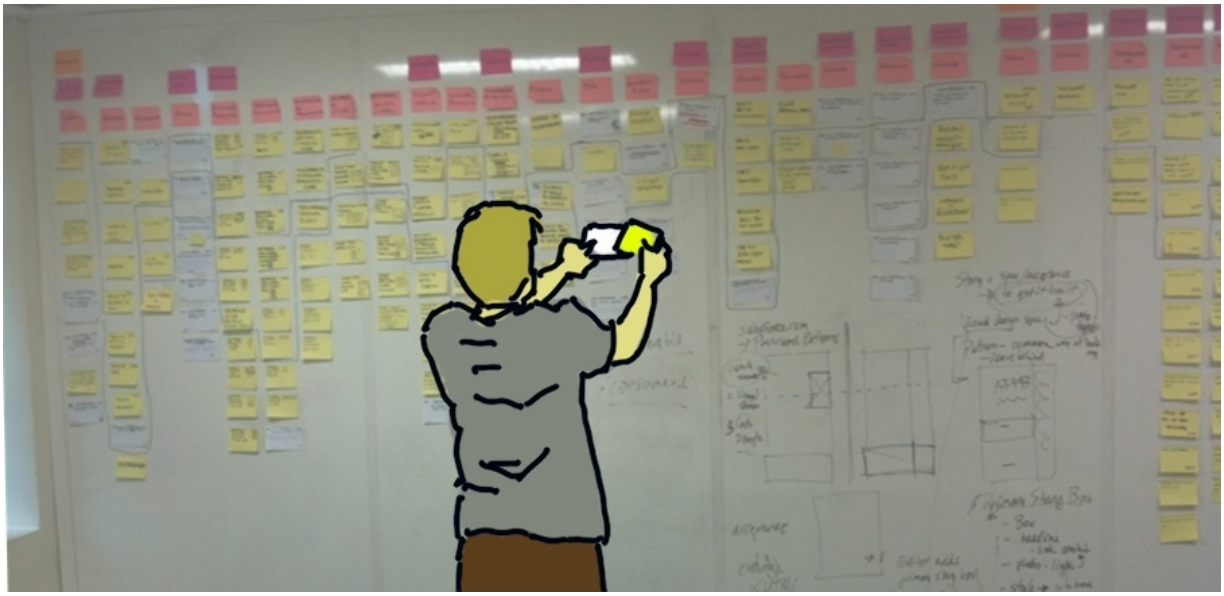
This story is about my friends at Globo.com, the largest media company in Brazil. Globo.com owns television and radio stations, produces made-for-TV movies and original programming, and publishes newspapers. It is a media monster in Brazil, and the largest Portuguese-language media company in the world.

Globo.com knows better than most organizations on the planet what unmovable deadlines really are. For example, it produces a cool version of a fantasy football game that’s revised and improved every year for World Cup soccer — the sport that most of the planet refers to as football. If Globo.com is late with development of that game, it doesn’t have the luxury of changing the release date. Why not? Because the rest of the world won’t reschedule the World Cup. Globo.com will produce features and content for the Olympics games that Brazil will host in 2016, and I can

guarantee you that it'll get it done in time — it has to. And it'll produce features and content for the release of numerous new television programs and reality TV shows. None of these things can be rescheduled if Globo.com is late. Globo.com must *always* finish on time. And, since that's the reality of its business, Globo.com is good at it. That's not because the company is faster than everyone else — sure, it's fast, but it's not *that* fast. It's because it's smart about doing less.

Mapping Helps Big Groups Build Shared Understanding

Take a look at this:



That's just a portion of a pretty big map built by leaders of eight teams from three different groups at Globo.com working together. The teams from Sports, News, and Entertainment built this together to think through the work they needed to do to rebuild, revamp, and renovate their underlying content management system. This is the system that drives all those news websites, sports websites, soap opera websites, features that help publicize and recruit people for reality TV shows, and much, much more. This massive system needs to be able to handle large quantities of video feeds, real-time

scores and election results, photographs, fast-breaking news stories, and much more. It has a lot to do, and it needs to look good doing it.

When I arrived at the Globo.com offices the day they built this map, the teams working together were about to fall into the *flat backlog trap*. Individual teams had prepared their respective prioritized backlogs. It was already clear there was a huge amount of work to do, and each team depended on the other. For instance, to get a good news site out would need not just the news team, but also all the other teams that built the foundational components that let the news website use photos, videos, real-time data, and lots of other things.

I sat down with them and reminded them of something they already knew: “I understand that you’re different teams because you’re focusing on different areas, but it’s a major revision of *one* content management system. You’ll have to release together. You can’t plan a release until you can see it all together. You’ve got to visualize all these dependencies.” They agreed and quickly went to work reorganizing their individual backlogs into a map. Within a few hours they built a map on the wall using sticky notes that told the story of their content management system.

I wasn’t in the room while team members worked together to build the map. But when I returned later in the day, I was amazed at how quickly they’d built it. They were pleased with themselves, and had every right to be. They’d made sense of several complex backlogs, organizing them into one coherent product story. And now each team could see where its work fit into the big picture.

Map for a product release across multiple teams to visualize dependencies.

ANATOMY OF A BIG MAP

Globo's map is good example of what a typical map looks like *after* you've framed, mapped, and explored lots of details.

The Backbone Organizes the Map

At the top of the map is the *backbone*, which sometimes has a couple of different levels. You might start with the basic flow of the story, which is one level. But, when it gets really long, it's useful to go up one more level to summarize things further. Later, I'll add some language about what I like to put at each level, but I'm reminded of an old friend of mine who told me to stop trying to come up with precise language. "It's just big things and little things," he'd tell me. And he's right.

The whole thing kind of looks like you extracted the spine out of a weird, Seussian animal. You've got this long backbone at the top with lots of irregularly spaced vertebrae, and these long ribs of varying length hanging down from it.



Map in Whole Deliverable Releases

This map was built by multiple teams at Globo. There are development teams responsible for video, and teams charged with the task of building the stuff on the backend that editors use to create and manage content. There are teams responsible for some of the underlying metadata and associations between data — that semantic markup junk that I can never quite grok. And there are people who handle the external presentation and how good this stuff looks when users and/or consumers see it. And still more people are looking after specific features relevant to news, sports, or entertainment.

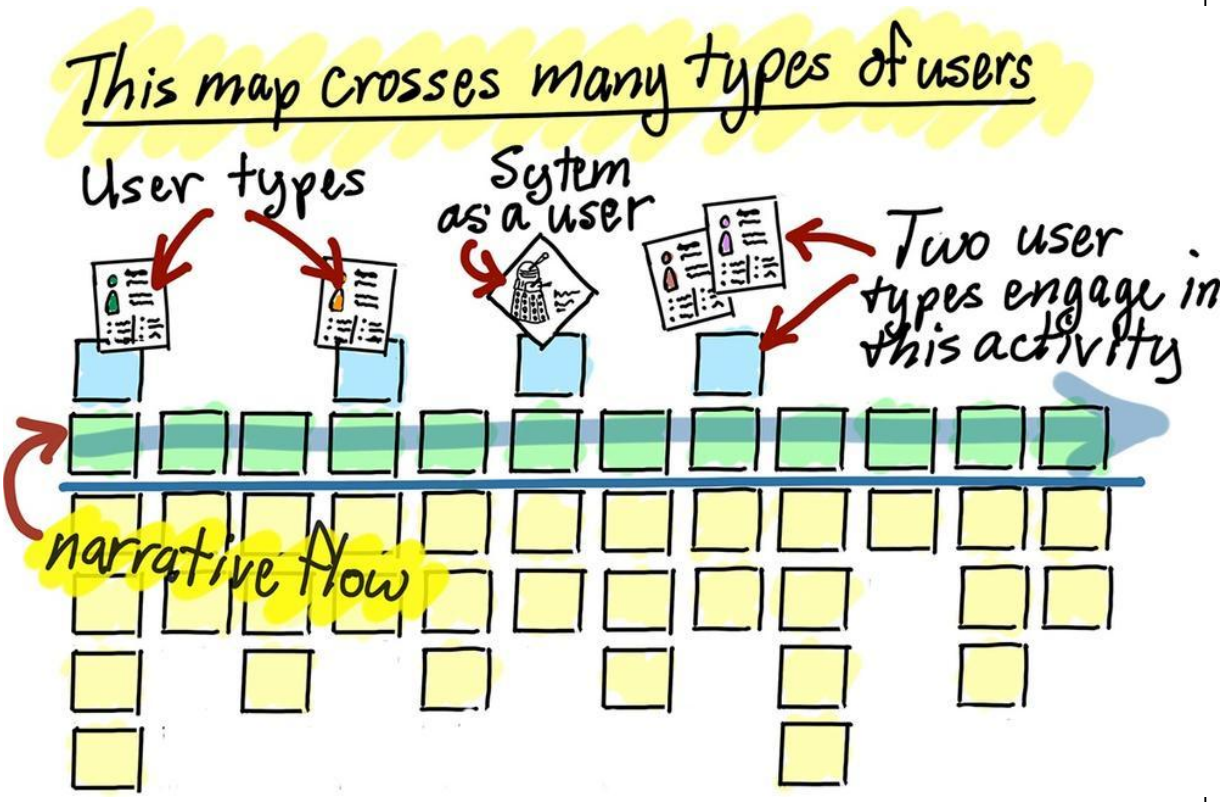
Multiple teams had to work together on this map because, for this major revision, no single team could release its part without the others. The teams built a single map because they needed to think through the release holistically.

Map in a Narrative Flow Across Many Users and Systems

The map started with people on the left, and the things they had to do to set up basic widgets for the screens that held news stories, pictures, and videos. There were other types of people who then combined those onto pages for soap opera or news websites. Then there were editors who added content to pages. This entire backbone tells the story of how lots of different people at Globo.com construct and manage content in its website.

When you read the backbone of the map from left to right, it tells a story about all the people who use the system and what they do in order to create and manage sites and content. The left-to-right order is what I call a *narrative flow*, which is an academic way of saying the order in which we'd tell the story. Of course, all these people are doing everything all at once, and sometimes things don't move in a perfect order, but we know that. We just put them in an order that helps us tell the story.

For this big system, that narrative flow has to *cut through many different users' and systems' stories*. I like to place stickies or simple persona thumbnails above the backbone so I can see who we're talking about at particular times in the story. And it's OK to anthropomorphize backend services or complex stuff the system does. My friends at SAP create fictitious personas for their systems and use pictures of R2D2 or C3PO from *Star Wars*.



Mapping Helps You Spot Holes in Your Story

When I talk with people who have built story maps, they'll tell me, "Every time we do this we find holes. We find things that we thought *another* team should be taking care of, but it didn't know. We find the necessary stuff in between the big important features that we'd forgot to talk about." By mapping together, Globo.com found some of that.

After you've envisioned the whole product or feature, it's easier to start to play the "What-About" game. That's where we start asking, "What about when this goes wrong?" Or "What about these other users?" Play What-About with any concern you have, and add stickies to the body of the map for feature ideas you'll need in order to address these things in your software. In Chapter 1, Gary played What-About to consider options and alternatives. When you do this with other teams, you'll find they're terrific at spotting problems that might arise where different systems connect to each other.

One of the criticisms people sometimes make about story mapping is that every time they sit down and create story maps, they end up with way too much. But it's my belief that we're just finding the stuff now that would have bitten us later on, and that's a good thing.

In the old-school approach to software development, when we'd find that new stuff later on — after we'd already estimated delivery time and committed to a delivery date — we'd call that new stuff *scope creep*. I personally believe that scope doesn't creep; understanding grows. And what happens when people build one of these story maps is that they find the holes in their understanding.

> Scope doesn't creep; understanding grows.

There's Always Too Much

When I left the content management teams at Globo.com, everything was wonderful — understanding was rampant and the teams knew what to do. However, when I came back to check in with them a couple days later, they were struggling again because they realized that there was so much work to be done, that it would likely take more than a year to accomplish everything on the map. And, of course, as savvy readers can appreciate, when software developers say it's going to take a year to get something done, they really mean two years. It's not because they're incompetent, or that they are calendar-challenged, it's just that estimating the time to do something we've never done before is something we suck at. And, by nature, we're often optimistic animals.

They said to me, "There's way too much. We have a lot to do here and it's going to take a long time."

"Do you have to do it all?" I asked.

They, of course, replied, "Yes, because it's all part of one, big content management system."

"But projects don't run that long around here," I replied. "I know your CEO and he's going to want to see results much faster — right?"

"Yes," they confirmed, "he wants to see something live in time for the upcoming Brazilian election in a few months!"

"Do you need *all* of this to go live for the election?" I asked.

As soon as I asked the question, I could see light bulbs click on. Of course they didn't need *everything*. Up 'til now, they'd been focused on identifying sequence and dependency assuming they'd need to build everything. They did, but it was really a question of when. They shifted their thinking to focus on outcomes.

> Focus on what you hope will happen outside the system to make decisions about what's inside the system.

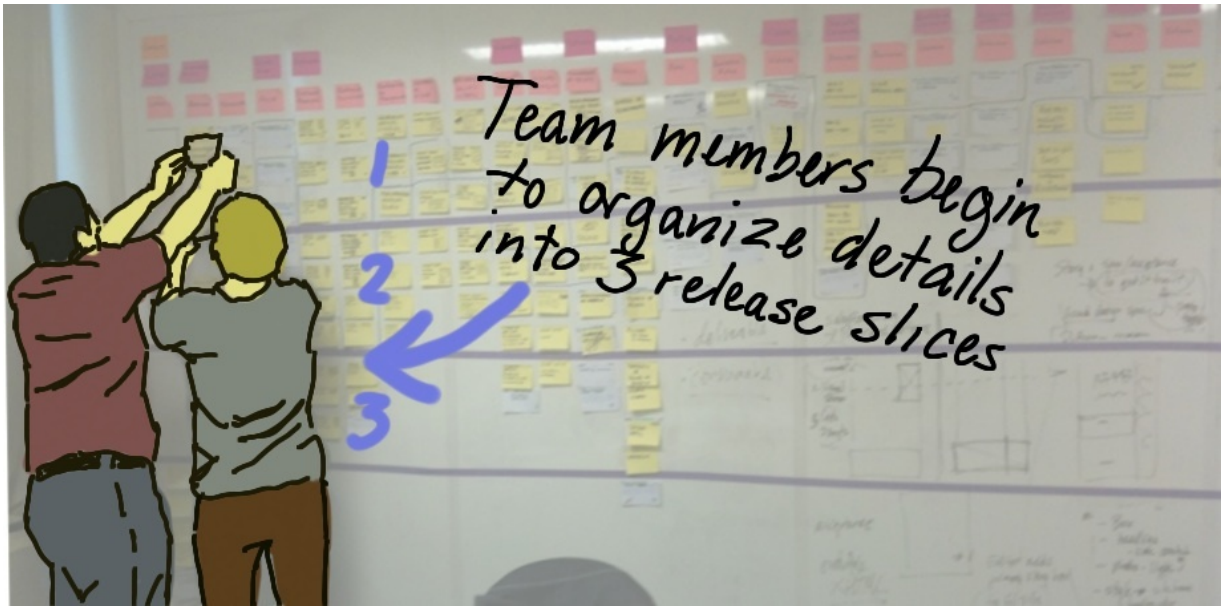
Globo.com focused on Brazilian elections. The teams thought specifically about the great *outcome* of successfully

impressing visitors, advertisers, and Globo's parent media company with newer, sexier styles of interactive content throughout the election. If they did that, they'd have a win.

This wasn't their first time facing down an impossible delivery date. They thought about it for a bit, and realized that they definitely needed to go live with the news website, and potentially some other things that supported it because that's where visitors and others would go to monitor Brazilian elections. Focusing on the news website meant paying attention to innovative ways to show real-time election data and breaking news stories faster. And, of course, there was a newer, up-to-date visual design overlaying everything.

Slice Out a Minimum Viable Product Release

The teams grabbed a roll of blue painter's tape and stretched lines across the map left to right to make horizontal slices. They then went to work moving cards up and down, above and below the blue lines to designate which things needed to be done in the first slice, and which could be done later.



The thought process worked a bit like this: *If we go live for the Brazilian elections, a lot of people in Brazil will see this thing.*

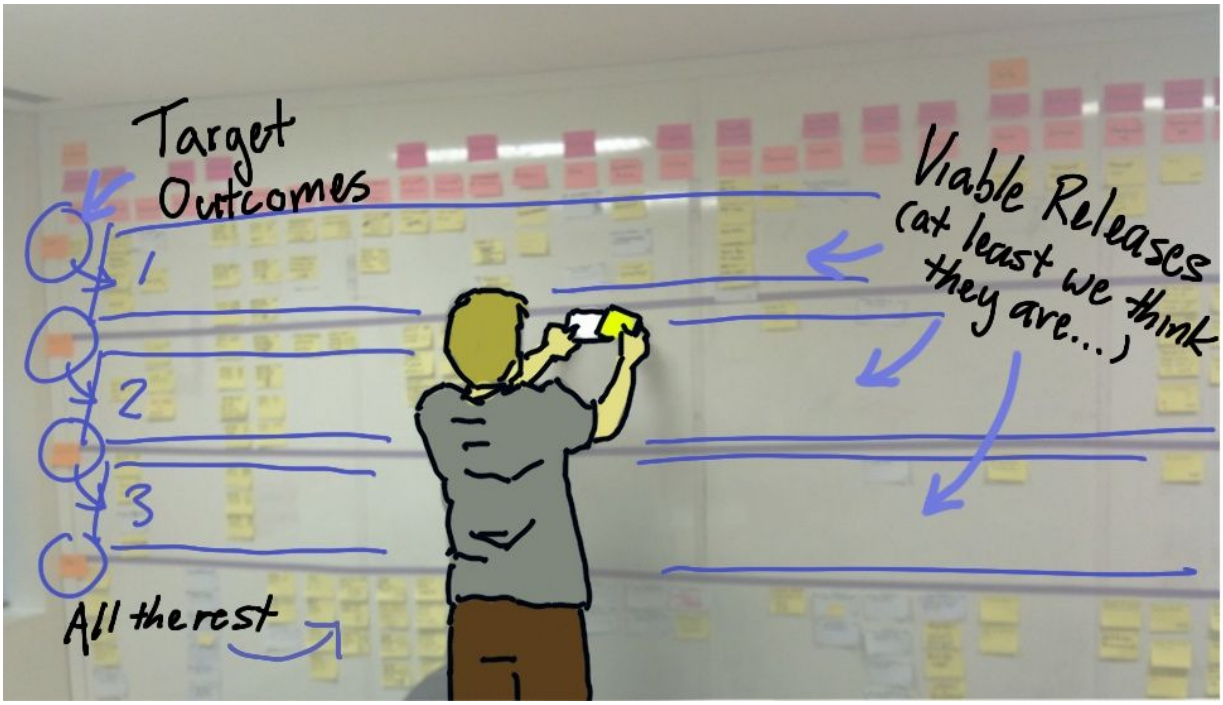
It’s going to land with a splash. It’s going to affect these websites of ours and we’ll look good. Everything in this slice is what users will need to be able to do after the software is released so they can make that splash.

> Focus on outcomes — what users need to do and see when the system comes out — and slice out releases that will get you those outcomes.

Slice Out a Release Roadmap

The map contained innovative things that would improve all Globo.com’s web properties. But it really was going to take a long time to get all of it done. Hitting the market window that the elections created would be too big an opportunity to miss. Focusing on that helped Globo identify the first release.

The teams then went to work thinking about what kinds of web properties and market events should anchor the next releases. They posted sticky notes to the left of each slice with a few words describing their intention for each release slice — their target outcomes. Then they continued to move cards up and down into their correct slices.



In the end, they had an incremental release strategy that let them tackle all the work they needed to do to replace the whole content management system over time, and in such a way that they saw real benefit with each release. If they read down the left edge of the map, they had a list of named releases, each with specific target outcomes. This is a *release roadmap*.

Notice how the list isn't a bunch of features. It's a list of real-world benefits — because, remember, your job isn't to build software, it's to change the world. The hard part is choosing which people you want to change the world for, and how.

> Focusing on specific target outcomes is the secret to prioritizing development work.

And the opposite is true as well; that is, if you don't know what your target outcomes are — the specific benefits you're trying to get — then prioritization is close to impossible.

Don't Prioritize Features — Prioritize Outcomes

Notice also how the Globo teams started with a big goal of replacing their entire content management system. Replacing the content management system is the output, the stuff they were going to deliver. Doing that would result in lots of positive outcomes. The secret to breaking down that really big chunk of output was to focus on a smaller, specific outcome.

Remember: behind outcomes are specific behavior changes for specific people engaged in specific activities. By focusing on the upcoming Brazilian elections, Globo chose to focus on the people who follow the news, especially those looking for up-to-the-minute election details. But, in placing its focus on those people, it left out soap opera lovers, sports lovers, and lots of other types of users. Those other people would have to be satisfied with the current site for a while longer. Remember, you can't please everyone all the time.

This Is Magic — Really, It Is

I may be easily impressed, but slicing is one of the coolest things about organizing software ideas into a story map.

Many times I, and the teams I've worked with, have placed all our ideas about the perfect product into a map and been overwhelmed by the amount of work we'd have if we created it all. It *all* seems important. But then we step back and think about the specific people who will use our product, and what they'll need to accomplish to be successful. We distill that into a sentence or two. Then we carve away everything we don't need, and we're shocked at how small our viable solution really is. It's *magic*.

Gary from Chapter 1 went on to do something similar with his product. He eventually narrowed focus to the band manager, the fan, and Mimi's internal administrator — because you've got to keep the site running. Gary chose to leave out venue managers and production musicians. In the end, what he realized is that by focusing on just those few people and the activity of promoting, he ended up with a great email promotion platform. So, for those of you who are Mimi users today, that's the experience you know.

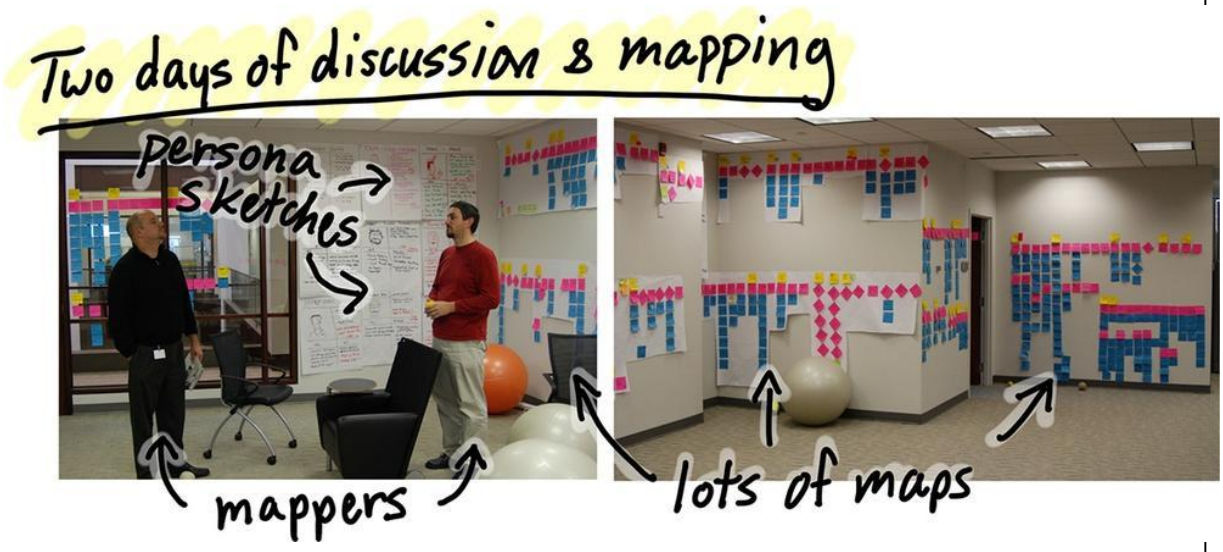
Externalizing our thinking in a big visible map makes all these steps easier. It makes it possible for lots of people to collaborate to accomplish it.

FINDING A SMALLER VIABLE RELEASE

Chris Shinkle, SEP

FORUM Credit Union is one of the nation's largest and most technologically progressive credit unions. Although the company had a competent and creative development culture, it approached SEP to build a new online banking system that rivaled existing commercial off-the-shelf (COTS) solutions. Its goals included adding capabilities for its mobile banking, personal finance management, and text banking.

SEP kicked off the engagement with a two-day, collaborative story-mapping discovery session that included outcomes, personas, and story mapping. The session helped facilitate a structured conversation to prioritize a large set of feature ideas. The outcomes and personas were not enough, however, to prioritize the stories. By the end of two days, the story map covered nearly two walls in the 1,000+ square-foot development area!



After the story map was constructed, SEP guided the FORUM stakeholders through a simple prioritization model:

Differentiator
: A feature that set them apart from their competition

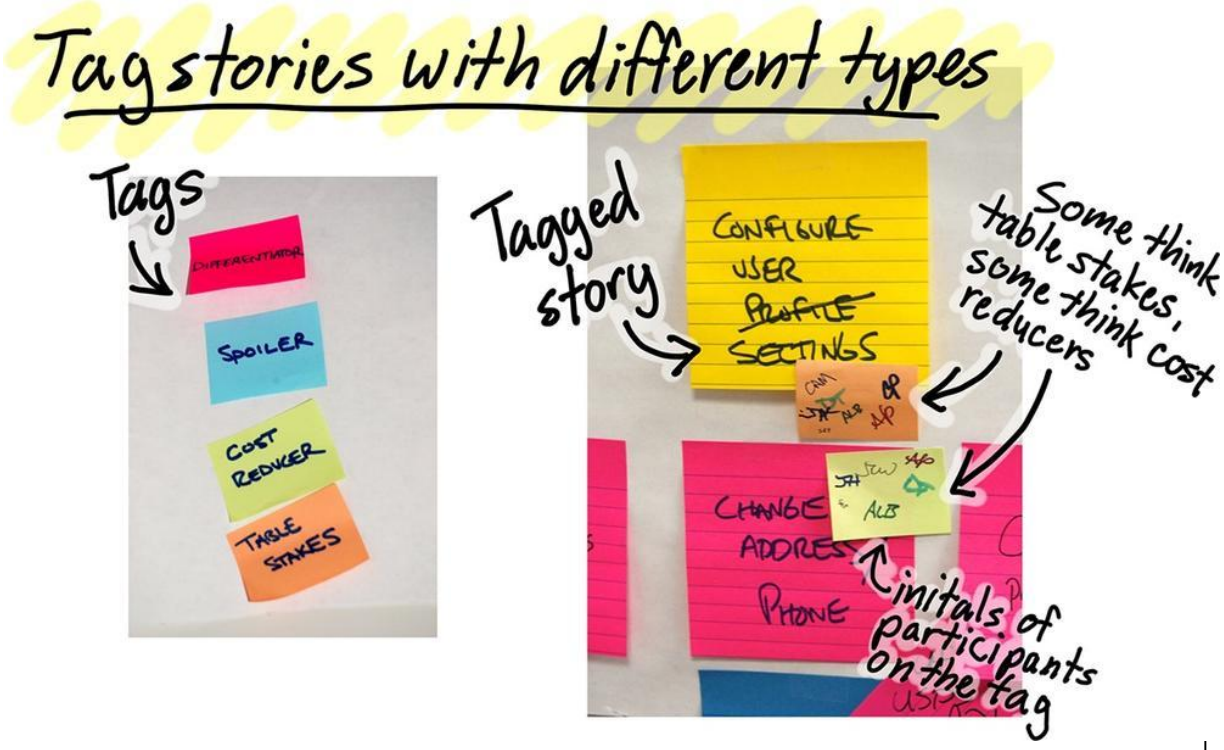
Spoiler
: A feature that is moving in on someone else's differentiator

Cost reducer
: A feature that reduces the organization costs

Table stakes
: A feature necessary to compete in the marketplace

SEP indicated each story's category using different color sticky notes. Interesting discussions emerged — in fact, some stakeholders' differentiators

were another stakeholder's table stakes. It was clear many of these conversations were happening for the first time! The story map with the prioritization model enabled conversations to happen that had not happened before. It helped guide the team to come to a shared understanding around priorities.



After labeling the stories, SEP used a voting system to help stakeholders converge the ideation and discussion into the most meaningful set of outcome-focused features. To everyone's surprise, several stories were deemed postpone-worthy or unnecessary. Rough calculations revealed several hundred thousand dollars in savings before one line of code was written.

When asked about using story mapping to kick off the project, Doug True, FORUM's CEO, said, "When we first kick-started this project with a story mapping process including the use of personas, I was skeptical. Specifically, I was concerned with the time invested to this softer side of the project. On the second day it clicked and the worthiness of the time materialized. In fact, now, I couldn't imagine pursuing a project of this scope and member-facing impact without such a process."

Why We Argue So Much About MVP

There's a term that's been kicking around in the software development industry for a long time: *minimum viable product*, or simply MVP. Frank Robinson is credited with originally coining the term *MVP*, but these days definitions

from Eric Ries and Steve Blank dominate. In spite of multiple smart people trying to define the term, everyone still seems confused — including me. Every organization I run into that uses that term means something slightly different. Even people in the same organization and the same conversation often mean different things.

Like most words in the dictionary, it has multiple meanings. I'm going to give you three definitions for the term: a bad one, and two good ones.

Here's the bad one:

> Minimum viable product is not the crappiest product you could possibly release.

And the MVP *isn't* the product that your users could use, but only in the simplest of circumstances, and only if they had a high threshold for pain. I commonly see organizations rationalizing their bad product decisions with the argument that someone could use the product, when it's clear to everyone involved that they probably wouldn't choose to.

If Globo.com had used this definition for slicing out its first release, it would have had a negative outcome. No one would have been impressed. The brand would have been hurt, and the company would have been worse off than if it had released nothing at all.

When we use the term *viable* when talking about a living organism, it means that an organism can survive in the world on its own without dying. And when we talk about software, we mean the same thing.

> The minimum viable product is the smallest product release that successfully achieves its desired outcomes.

I like this definition best. *Minimum* is a subjective term. So be specific about who it's subjective to — because it's not you. Be specific about who your customers and users are, and what they need to accomplish. What's *minimum* to them? I promise you, it'll help the conversation hugely. It's still a

tough conversation. But the alternative is the “HiPPO” method — the “highest paid person’s opinion.” That one sucks worse.

The term I prefer these days is *minimum viable solution*. Most of the things I work with organizations on aren’t whole, new products. They’re new features or capabilities, or improvements on features already out there. So the term *solution* seems to make more sense. So let me revise my definition:

> The minimum viable solution is the smallest solution release that successfully achieves its desired outcomes.

Here comes the hard part…

We’re just guessing.

When we slice out a bunch of software functionality and call it a minimum viable solution, we don’t really know if it is.

The problem with outcomes is that you can’t really observe them until things come out. When you slice out a release, you’re forced to hypothesize what will happen. You might have to guess about what customers will buy your product, what users will choose to use it, if they can use it, and what’s feasible to build in the time you have. You’re forced to guess at how much will make them happy. That’s a lot of guessing.

This sucks, because if you guess too low, well, that’s less than minimal, and you’ve failed. If you guess too high, which many people do to hedge their bets, then you’ve spent too much money and often risk not getting things done at all. And worst of all, you could be just dead wrong, and no amount of what you ship will matter at all.

It’s no wonder that the “crappiest product you could possibly release” definition still thrives. Because that’s the one we don’t need to guess about.

The New MVP Isn’t a Product at All!

I know that some of you may have been feeling progressively twitchy over the last couple of chapters. You may have

thought to yourself, *Jeff's overlooking the most important thing of all!* And you might be right. Some of the most important things you can discuss during story and story map conversations are:

- What are our biggest, riskiest assumptions? Where is the uncertainty?
- What could I do to learn something that would replace risks or assumptions with real information?

This leads me to my third definition of MVP, as popularized by Eric Ries in his book *The Lean Startup* (Crown Business). Eric learned the hard way, as most of us do, that we're just guessing. Eric worked for a company that released a product that it thought was viable, but it was wrong. Intelligently, he changed his strategy to focus on learning — to focus on validating all those assumptions the company had made in its first MVP release. Eric makes the important point that we need to create smaller experiments, prototypes that test our hypothesis about what's minimal and viable. And if you adopt Eric's way of thinking, which you should, your first product should really be an experiment — and the one after that, and the one after that, until you really prove that you've got the right product.

> A minimal viable product is also the smallest thing you could create or do to prove or disprove an assumption.

While it was pretty cool that the folks at Globo.com were able to create a plan to build less, they weren't fooling themselves. They knew there was lots left to learn to prove their assumptions were good. From here they and everyone else need to create a plan to learn more. And that's where we'll pick up our story in the next chapter.

OceanofPDF.com